



Lab 1: Getting Started with RISC-V (Assembly Language) in VS Code

GITHUB REPO LINK: [Click Here](#)

Task 1:

```
M Abdullah Khan@DESKTOP-BPMTBI1 MINGW64 ~
$ git --version
git version 2.47.1.windows.2

M Abdullah Khan@DESKTOP-BPMTBI1 MINGW64 ~
$ git config --global
error: no action specified

M Abdullah Khan@DESKTOP-BPMTBI1 MINGW64 ~
$ git config --global user.name
Muhammad Abdullah Khan

M Abdullah Khan@DESKTOP-BPMTBI1 MINGW64 ~
$ git config --global user.email
abdullahmawaz43@gmail.com

M Abdullah Khan@DESKTOP-BPMTBI1 MINGW64 ~
$ git config --global --list
core.editor="C:\Users\M Abdullah Khan\AppData\Local\Programs\Microsoft VS Code\bin\code" --wait
user.name=Muhammad Abdullah Khan
user.email=abdullahmawaz43@gmail.com
safe.directory=%(prefix)///wsl.localhost/Ubuntu/root/lab02-Khanx007

M Abdullah Khan@DESKTOP-BPMTBI1 MINGW64 ~
$ |
```

```
Administrator@Laptop MINGW64 ~
$ git --version
git version 2.51.1.windows.1

Administrator@Laptop MINGW64 ~
$ git config --global --list
user.email=@syedsualehabbas

Administrator@Laptop MINGW64 ~
$ git config --global user.name "syedsualehabbas"

Administrator@Laptop MINGW64 ~
$ git config --global user.email "sualehabbas.6981@gmail.com"

Administrator@Laptop MINGW64 ~
$ git config --global --list
user.email=sualehabbas.6981@gmail.com
user.name=syedsualehabbas

Administrator@Laptop MINGW64 ~
$
```



```
M Abdullah Khan@DESKTOP-BPMTBI1 MINGW64 ~/OneDrive/Documents/CA Labs
$ git clone https://github.com/khan-slayer/Lab01-Git-MuhammadAbdullahKhan.git
Cloning into 'Lab01-Git-MuhammadAbdullahKhan'...
warning: You appear to have cloned an empty repository.

M Abdullah Khan@DESKTOP-BPMTBI1 MINGW64 ~/OneDrive/Documents/CA Labs
$ cd Lab01-Git-MuhammadAbdullah_SyedSua1eh
bash: cd: Lab01-Git-MuhammadAbdullah_SyedSua1eh: No such file or directory

M Abdullah Khan@DESKTOP-BPMTBI1 MINGW64 ~/OneDrive/Documents/CA Labs
$ cd Lab01-Git-MuhammadAbdullahKhan/

M Abdullah Khan@DESKTOP-BPMTBI1 MINGW64 ~/OneDrive/Documents/CA Labs/Lab01-Git-MuhammadAbdullahKhan (main)
$ touch hello.txt

M Abdullah Khan@DESKTOP-BPMTBI1 MINGW64 ~/OneDrive/Documents/CA Labs/Lab01-Git-MuhammadAbdullahKhan (main)
$ nano bye.txt

M Abdullah Khan@DESKTOP-BPMTBI1 MINGW64 ~/OneDrive/Documents/CA Labs/Lab01-Git-MuhammadAbdullahKhan (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    bye.txt
    hello.txt

nothing added to commit but untracked files present (use "git add" to track)

M Abdullah Khan@DESKTOP-BPMTBI1 MINGW64 ~/OneDrive/Documents/CA Labs/Lab01-Git-MuhammadAbdullahKhan (main)
$ git add bye.txt
warning: in the working copy of 'bye.txt', LF will be replaced by CRLF the next time Git touches it

M Abdullah Khan@DESKTOP-BPMTBI1 MINGW64 ~/OneDrive/Documents/CA Labs/Lab01-Git-MuhammadAbdullahKhan (main)
$ git commit -m "Added bye.txt"
[main (root-commit) a1f119f] Added bye.txt
 1 file changed, 1 insertion(+)
 create mode 100644 bye.txt
a
```

[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Security](#) [Insights](#) [Settings](#)

 **Lab01-Git-MuhammadAbdullahKhan** Private Watch 0

 main

 1 Branch

 0 Tags





[Code](#)

 **khan-slayer** Added bye.txt a1f119f · 11 minutes ago 1 Commit

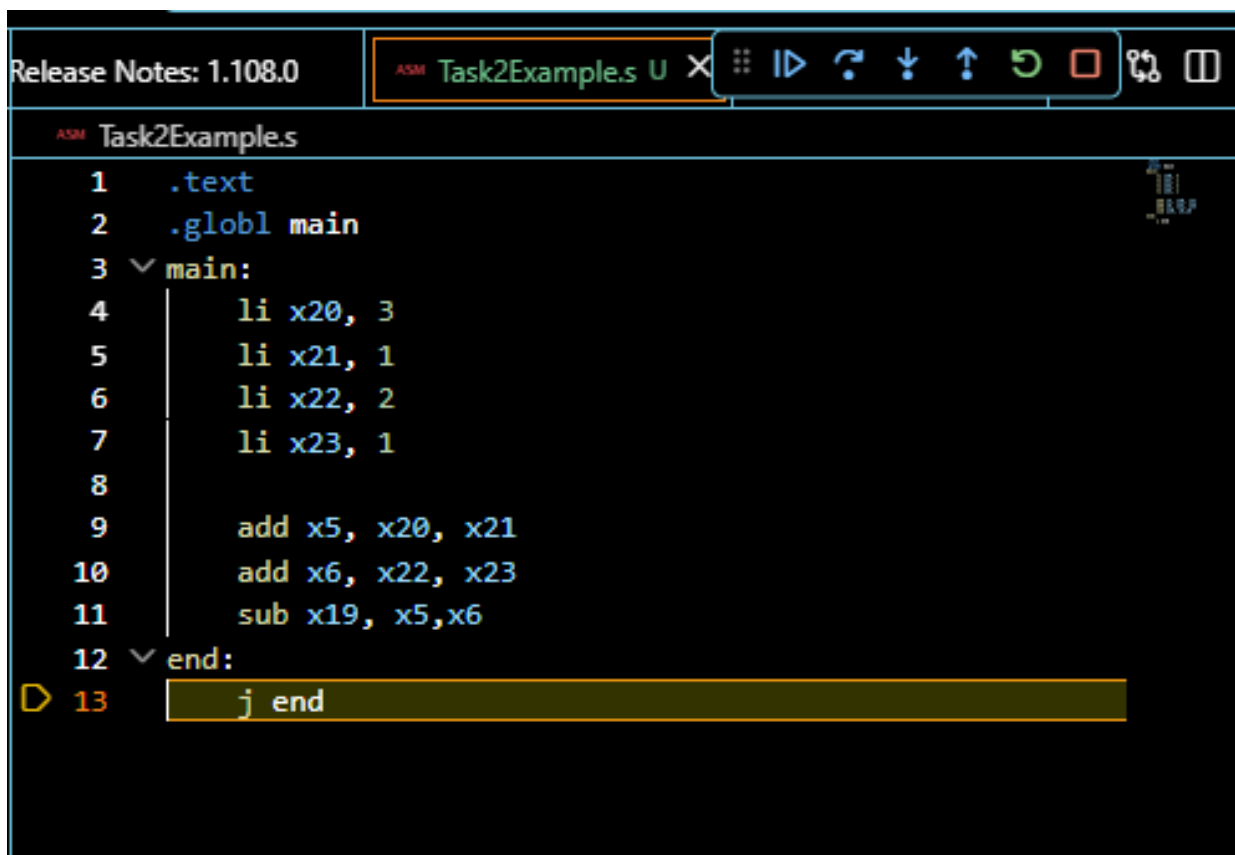
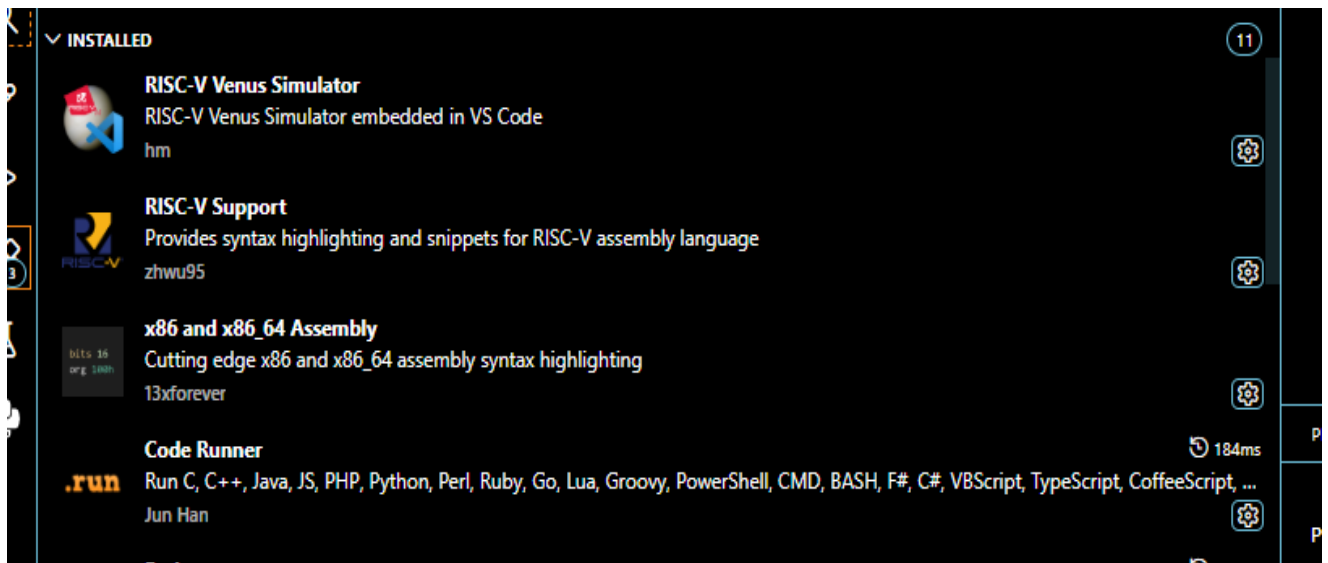
 bye.txt Added bye.txt 11 minutes ago

 [README](#)

UNIP

Sort View						
	Name	Status	Date modified	Type	Size	
	bye		1/13/2026 3:06 PM	Text Document	1 KB	
llah - Per	hello		1/13/2026 3:05 PM	Text Document	0 KB	
llah - Hat						

Task 02: Setting Up VS Code (RISC-V Simulation Environment)



VIEWING REGISTER VALUES:-



```
x04 (t0) = 0x00000000
x05 (t0) = 0x00000004
x06 (t1) = 0x00000003
x07 (t2) = 0x00000000
x08 (s0) = 0x00000000
x09 (s1) = 0x00000000
x10 (a0) = 0x00000000
x11 (a1) = 0x00000000
x12 (a2) = 0x00000000
x13 (a3) = 0x00000000
x14 (a4) = 0x00000000
x15 (a5) = 0x00000000
x16 (a6) = 0x00000000
x17 (a7) = 0x00000000
x18 (s2) = 0x00000000
x19 (s3) = 0x00000001
x20 (s4) = 0x00000003
x21 (s5) = 0x00000001
x22 (s6) = 0x00000002
x23 (s7) = 0x00000001
x24 (s8) = 0x00000000
```

EXAMPLE # 2:

The screenshot shows a debugger interface with two main panels. The left panel, titled 'VARIABLES', lists integer registers x07 through x23 with their current values in hexadecimal. The right panel, titled 'Task2Example.s', shows the assembly code for a program. The code includes instructions for loading values into registers, adding, subtracting, and incrementing, and finally jumping to the end of the program.

VARIABLES

- x07 (t2) = 0x00000000
- x08 (s0) = 0x00000000
- x09 (s1) = 0x00000000
- x10 (a0) = 0x00000000
- x11 (a1) = 0x00000000
- x12 (a2) = 0x00000000
- x13 (a3) = 0x00000000
- x14 (a4) = 0x00000000
- x15 (a5) = 0x00000000
- x16 (a6) = 0x00000000
- x17 (a7) = 0x00000000
- x18 (s2) = 0x00000000
- x19 (s3) = 0x00000001
- x20 (s4) = 0x00000003
- x21 (s5) = 0x00000001
- x22 (s6) = 0x00000006
- x23 (s7) = 0x00000001

Task2Example.s

```
1 .text
2 .globl main
3 main:
4     li x20, 3
5     li x21, 1
6     li x22, 2
7     li x23, 1
8
9     add x5, x20, x21
10    add x6, x22, x23
11    sub x19, x5, x6
12
13    addi x22, x22, 4
14 end:
15 j end
```

Figure 1.4: RISC-V Memory

Task 3

Convert the following statement to RISC V. You can use the same registers as given in

```
1  int a = 5;  
2  int b = 0 + 0;  
3  a = b + 32;  
4  int d = (a + b) - 5;  
5  int e = (((a - d) + (b - a)) + d);  
6  e = a + b + d + e;
```

Task 3

Code

```
.text  
.globl main  
main:  
    li x22, 5 # a = 5  
    add x23, x0, x0 #b= 0+0  
    addi x22, x23, 32 #a = b+32  
    add x24, x22, x23 # temp= a+b  
    addi x24, x24, -5 # d = temp -5  
    sub x25, x22, x24 # e_temp = a-d  
    sub x26, x23, x22 # temp = b-a  
    add x25, x25, x26 #e_temp = e_temp + temp  
    add x25, x25, x24 # e = e_temp + d  
    add x25, x25, x24 # e = e + d  
    add x25, x25, x23 # e = e+b  
    add x25, x25, x22 # e = e+a  
end:  
    j end
```

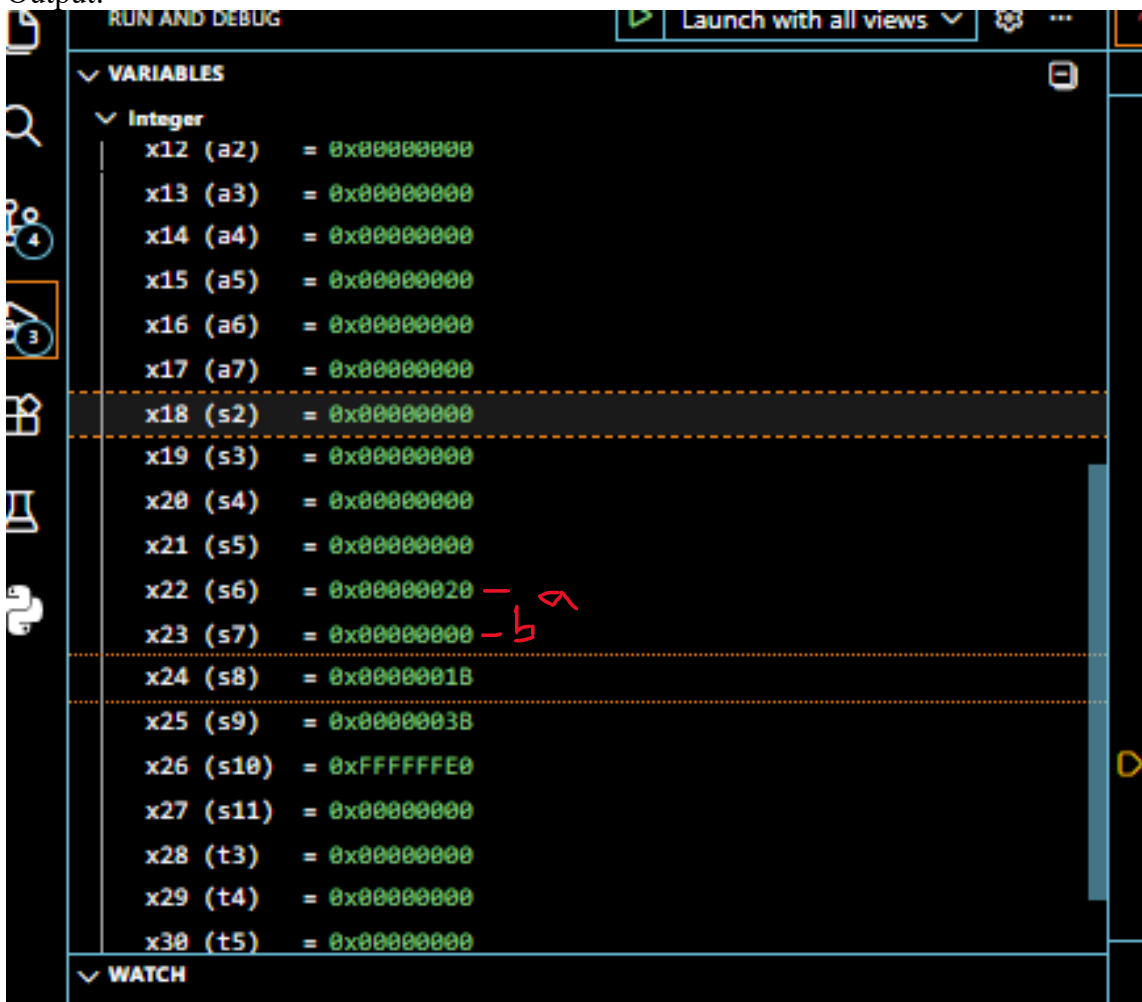
Output

Desired output:



Variable	Final Value
a	32
b	0
d	27
e	59

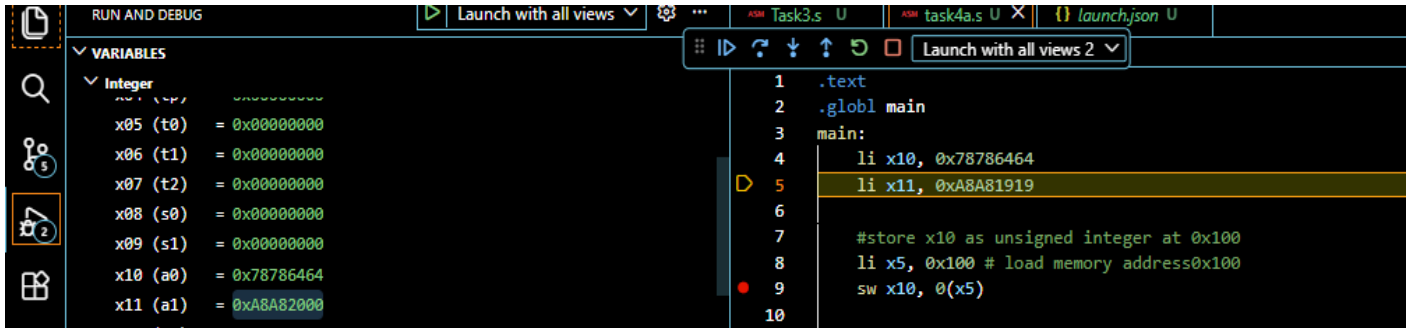
Output:



Hence same as desired output.

Task 4a

Initialize the register x10 and x11 with values 0x78786464, 0xA8A81919, respectively manually.



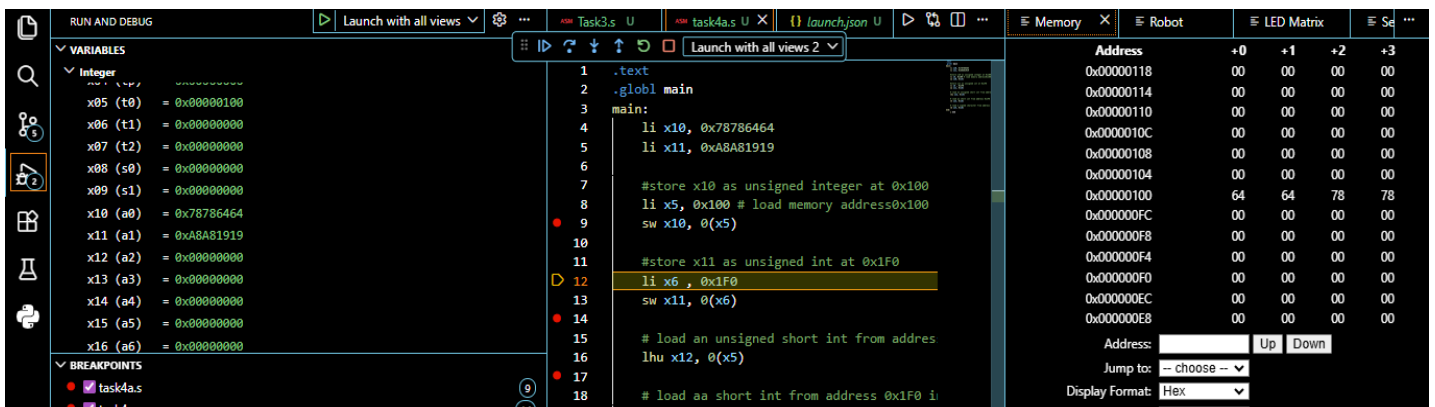
```

1 .text
2 .globl main
3 main:
4     li x10, 0x78786464
5     li x11, 0xA8A81919
6
7     #store x10 as unsigned integer at 0x100
8     li x5, 0x100 # load memory address 0x100
9     sw x10, 0(x5)
10

```

Write the RISC-V assembly code for each item below. Try guessing the result in each destination before executing the instruction and corroborate it after execution:

Store x10 as unsigned integer at address 0x100.

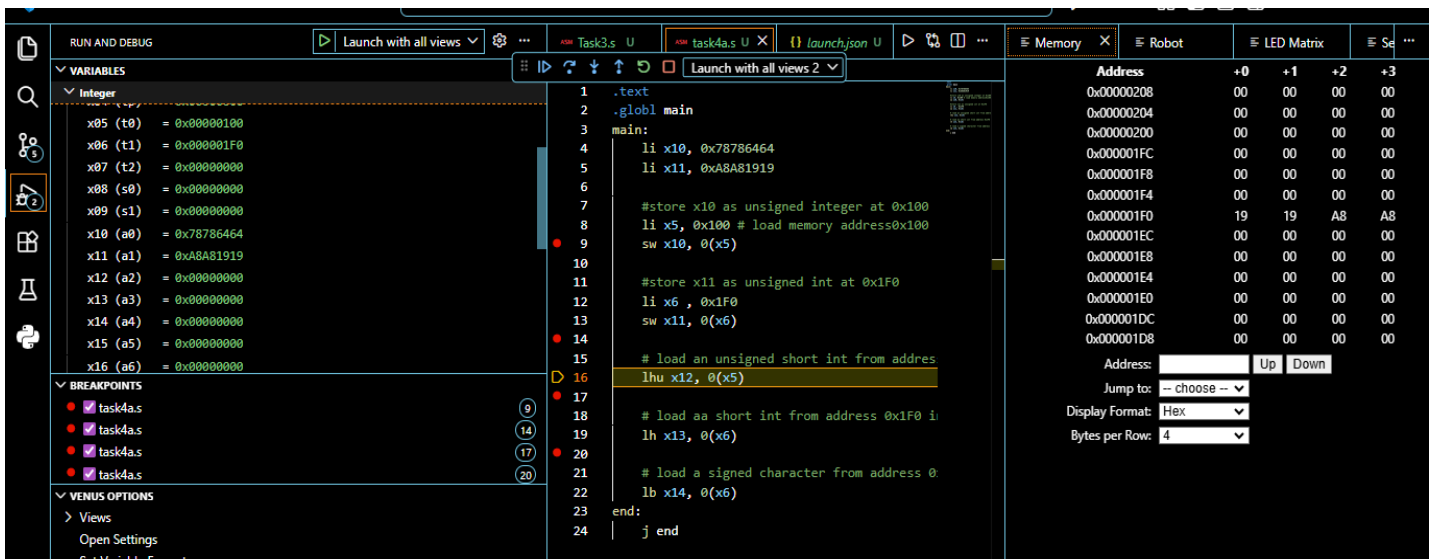


```

1 .text
2 .globl main
3 main:
4     li x10, 0x78786464
5     li x11, 0xA8A81919
6
7     #store x10 as unsigned integer at 0x100
8     li x5, 0x100 # load memory address 0x100
9     sw x10, 0(x5)
10
11    #store x11 as unsigned int at 0x1F0
12    li x6, 0x1F0
13    sw x11, 0(x6)
14
15    # load an unsigned short int from address
16    lhu x12, 0(x5)
17
18    # load aa short int from address 0x1F0 i
19

```

Store x11 as unsigned integer at address 0x1F0.

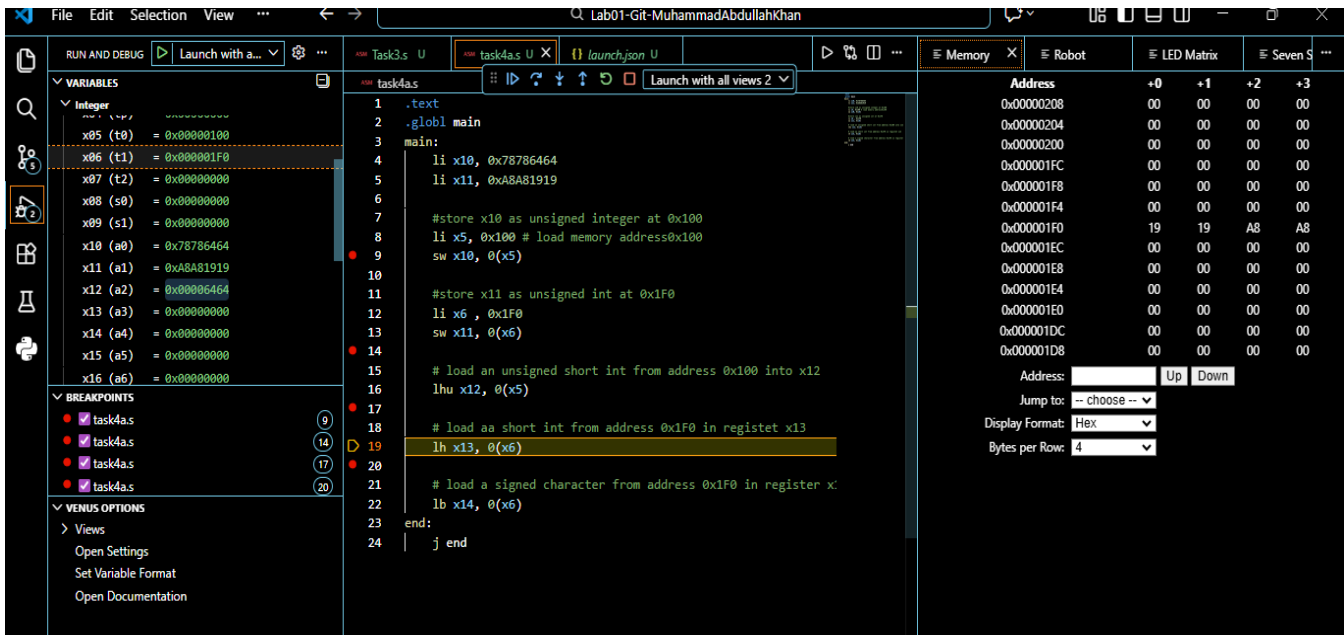


```

1 .text
2 .globl main
3 main:
4     li x10, 0x78786464
5     li x11, 0xA8A81919
6
7     #store x10 as unsigned integer at 0x100
8     li x5, 0x100 # load memory address 0x100
9     sw x10, 0(x5)
10
11    #store x11 as unsigned int at 0x1F0
12    li x6, 0x1F0
13    sw x11, 0(x6)
14
15    # load an unsigned short int from address
16    lhu x12, 0(x5)
17
18    # load aa short int from address 0x1F0 i
19    lh x13, 0(x6)
20
21    # load a signed character from address 0
22    lb x14, 0(x6)
23
24    end:
25    j end

```

Load an unsigned short integer (two bytes) from address 0x100 in x12.



The screenshot shows the Lab01-Git-MuhammadAbdullahKhan IDE. The assembly code for task4.s is displayed in the center pane. The code is as follows:

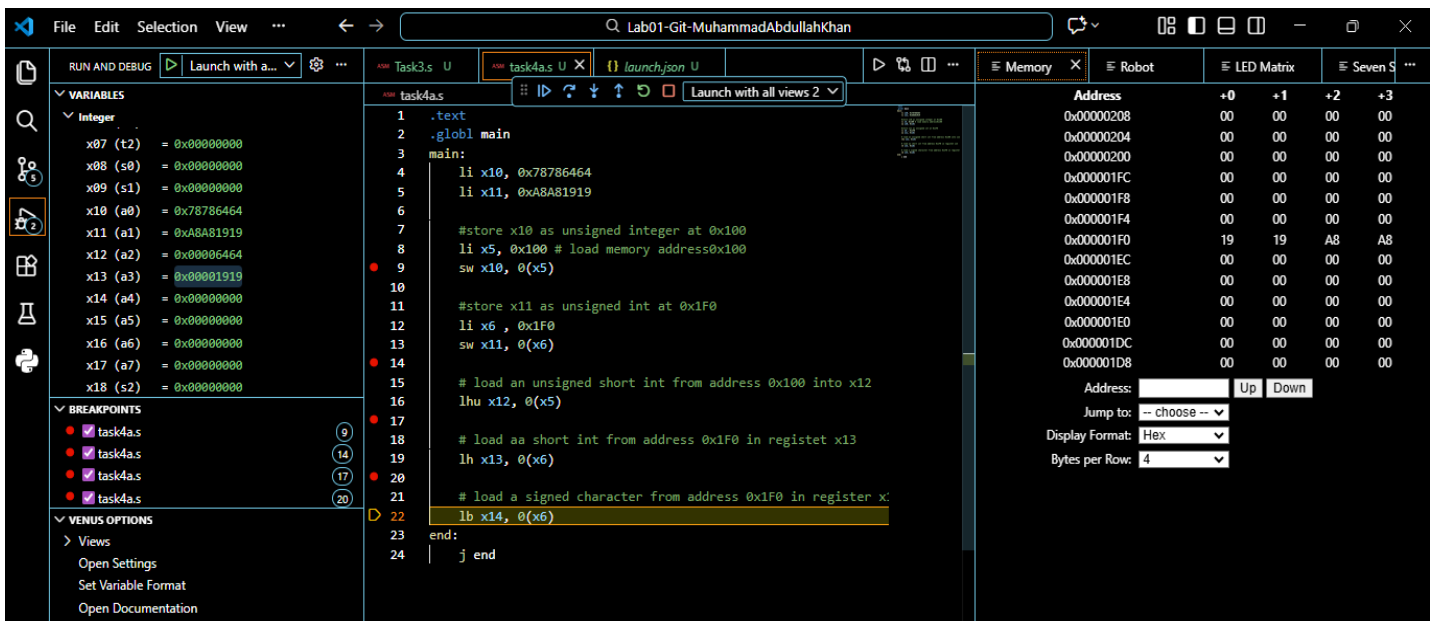
```

1 .text
2 .globl main
3 main:
4     li x10, 0x78786464
5     li x11, 0xA8A81919
6
7     #store x10 as unsigned integer at 0x100
8     li x5, 0x100 # load memory address 0x100
9     sw x10, 0(x5)
10
11    #store x11 as unsigned int at 0x1F0
12    li x6, 0x1F0
13    sw x11, 0(x6)
14
15    # load an unsigned short int from address 0x100 into x12
16    lhu x12, 0(x5)
17
18    # load aa short int from address 0x1F0 in registet x13
19    lh x13, 0(x6)
20
21    # load a signed character from address 0x1F0 in register x14
22    lb x14, 0(x6)
23 end:
24 j end

```

The memory window on the right shows the address 0x00001F0 containing the value 19 19 A8 A8. The display format is set to Hex and the bytes per row is 4.

Load a short integer from address 0x1F0 in register x13.



The screenshot shows the Lab01-Git-MuhammadAbdullahKhan IDE. The assembly code for task4.s is displayed in the center pane. The code is as follows:

```

1 .text
2 .globl main
3 main:
4     li x10, 0x78786464
5     li x11, 0xA8A81919
6
7     #store x10 as unsigned integer at 0x100
8     li x5, 0x100 # load memory address 0x100
9     sw x10, 0(x5)
10
11    #store x11 as unsigned int at 0x1F0
12    li x6, 0x1F0
13    sw x11, 0(x6)
14
15    # load an unsigned short int from address 0x100 into x12
16    lhu x12, 0(x5)
17
18    # load aa short int from address 0x1F0 in registet x13
19    lh x13, 0(x6)
20
21    # load a signed character from address 0x1F0 in register x14
22    lb x14, 0(x6)
23 end:
24 j end

```

The memory window on the right shows the address 0x00001F0 containing the value 19 19 A8 A8. The display format is set to Hex and the bytes per row is 4.

Load a signed character from address 0x1F0 in register x14.



The screenshot shows the RISC-V IDE interface. On the left, the 'VARIABLES' panel lists registers x07 through x18 with their current values. The 'BREAKPOINTS' panel shows four breakpoints set at lines 9, 14, 17, and 20 of 'task4a.s'. The 'VENUS OPTIONS' panel shows 'Views', 'Open Settings', and 'Set Variable Format'. The main editor displays assembly code for 'task4a.s'.

```

1  .text
2  .globl main
3  main:
4      li x10, 0x78786464
5      li x11, 0xA8A81919
6
7      #store x10 as unsigned integer at 0x100
8      li x5, 0x100 # load memory address 0x100
9      sw x10, 0(x5)
10
11     #store x11 as unsigned int at 0x1F0
12     li x6, 0x1F0
13     sw x11, 0(x6)
14
15     # load an unsigned short int from address 0x100 into x12
16     lhu x12, 0(x5)
17
18     # load aa short int from address 0x1F0 in register x13
19     lh x13, 0(x6)
20
21     # load a signed character from address 0x1F0 in register x14
22     lb x14, 0(x6)
23 end:
24 j end
  
```

Task 4b – - Loop unrolling

Assume there are three character arrays a, b, and c located at addresses 0x100, 0x200, 0x300 respectively.

```

for (int i=0 ; i<4; i++ )
c [ i ]=a [ i ]+b [ i ] ; # c [ 0 ]=a [ 0 ]+b [ 0 ] ;
  
```

Write equivalent RISC-V code for the piece of code given. You have not studied loops yet, but the above code is manageable without loop instructions. Also assume that A is a character array, B is a short array, and C is an unsigned integer array.

Code

```
ASM task4b.s U X
ASM task4b.s
1  .text
2  .globl main
3  main:
4      li x10, 0x100 #a
5      li x11, 0x200 #b
6      li x12, 0x300 #c
7
8      ## first iteration
9      lb  x13, 0(x10)    # a[0]
10     lh  x14, 0(x11)    # b[0]
11     add x15, x13, x14 # a[0]+b[0]
12     sw  x15, 0(x12)    # c[0]
13
14     ## second iteration
15     lb  x13, 1(x10)    # a[1]
16     lh  x14, 2(x11)    # b[1]
17     add x15, x13, x14 # a[1]+b[1]
18     sw  x15, 4(x12)    # c[1]
19
20     ## third iteration
21     lb  x13, 2(x10)    # a[2]
22     lh  x14, 4(x11)    # b[2]
23     add x15, x13, x14 # a[2]+b[2]
24     sw  x15, 8(x12)    # c[2]
25
26     ## fourth iteration
27     lb  x13, 3(x10)    # a[3]
28     lh  x14, 6(x11)    # b[3]
29     add x15, x13, x14 # a[3]+b[3]
30     sw  x15, 12(x12)   # c[3]
31
32     end:
33     j  end
Launch with all
```



Output

```
x07 (t2) = 0x00000000
x08 (s0) = 0x00000000
x09 (s1) = 0x00000000
x10 (a0) = 0x00000100
x11 (a1) = 0x00000200
x12 (a2) = 0x00000300
x13 (a3) = 0x00000000
x14 (a4) = 0x00000000
x15 (a5) = 0x00000000
```

Hence all addresses are stored in the registers.

**Assessment Rubric****Lab 1: Getting Started with RISC-V (Assembly Language) in VS Code**

Name: Syed Sualeh, Abdullah Khan	Student ID: 10303, 10022	section*: T4
---	---------------------------------	---------------------

Points Distribution

	Task No.	LR 2 Code	LR 5 Results
In - Lab	Task 1	/0	/15
	Task 2	/0	/15
	Task 3	/10	/5
	Task 4a	/10	/5
	Task 4b	/10	/10
Total Points: 100		/30	/50
CLO Mapped		CLO 2	

Affective Domain Rubric		Points	CLO Mapped
AR7	rt Submission & Git Upload	/10 & /10	CLO 2

CLO	Total Points	Points Obtained
2	100	
Total	100	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.

